



PARAMETIZACION EVALUACION DEL SOFTWARE

PROFESOR

NELSON AGUSTO FORERO

ESTUDIANTE

NICOLE VANESSA ARDILA RODRIGUEZ

TALLER QA WEB — SELENIUM, CYPRESS, OWASP ZAP, LOADRUNNER Y HOTJAR

AÑO

2026

INTRODUCCIÓN

El aseguramiento de la calidad de software (Software Quality Assurance — SQA) es un proceso fundamental dentro del desarrollo de aplicaciones modernas, ya que permite garantizar que los sistemas funcionen correctamente, sean seguros, soporten altos volúmenes de usuarios y ofrezcan una experiencia adecuada para el usuario final. En la actualidad, las aplicaciones web enfrentan múltiples desafíos relacionados con rendimiento, seguridad, automatización y usabilidad, por lo que es necesario implementar diferentes tipos de pruebas que permitan validar el correcto funcionamiento de los sistemas.

Las pruebas de software no solo se enfocan en encontrar errores funcionales, sino también en identificar vulnerabilidades de seguridad, medir el comportamiento del sistema bajo carga y analizar cómo interactúan los usuarios con las plataformas. Para ello, existen herramientas especializadas que facilitan la automatización y el análisis de diferentes escenarios de prueba.

En el presente taller se trabajó con herramientas ampliamente utilizadas en el ámbito profesional del aseguramiento de calidad de software, entre ellas Selenium y Cypress para automatización funcional, OWASP ZAP para pruebas de seguridad, LoadRunner para pruebas de rendimiento y Hotjar para análisis de usabilidad y comportamiento del usuario.

Selenium permitió automatizar procesos web mediante scripts en Python, facilitando la validación automática de flujos como el inicio de sesión. Por otra parte, Cypress se utilizó para realizar pruebas end-to-end en aplicaciones web modernas, destacándose por su facilidad de configuración y depuración. En cuanto a seguridad, OWASP ZAP permitió ejecutar análisis automatizados con el objetivo de identificar posibles vulnerabilidades presentes en aplicaciones web. Adicionalmente, se estudiaron conceptos relacionados con pruebas de carga utilizando LoadRunner y análisis de experiencia de usuario mediante Hotjar.

El desarrollo de este taller permitió comprender la importancia de aplicar distintos enfoques de pruebas dentro del ciclo de vida del software, integrando aspectos funcionales, rendimiento, seguridad y usabilidad. De esta manera, se evidencia cómo el uso combinado de herramientas de QA contribuye a mejorar la calidad, estabilidad y confiabilidad de las aplicaciones web modernas.

JUSTIFICACIÓN

La implementación de pruebas de software es indispensable en el desarrollo de aplicaciones web debido al crecimiento de la complejidad tecnológica y al aumento de riesgos asociados con fallos funcionales, vulnerabilidades de seguridad y problemas de rendimiento.

Actualmente, las organizaciones requieren sistemas confiables que garanticen disponibilidad, estabilidad y una adecuada experiencia para los usuarios.

Las herramientas de automatización y análisis permiten reducir errores humanos, optimizar tiempos de validación y detectar problemas antes de que las aplicaciones sean utilizadas en entornos reales. Además, el uso de metodologías de aseguramiento de calidad facilita el mantenimiento y evolución continua de los sistemas.

Este taller resulta importante porque permite adquirir conocimientos básicos sobre herramientas utilizadas en entornos reales de QA y DevOps, fortaleciendo competencias relacionadas con automatización de pruebas, análisis de seguridad, pruebas de rendimiento y evaluación de experiencia de usuario. Asimismo, proporciona una visión integral sobre la calidad de software y su impacto en el correcto funcionamiento de aplicaciones web modernas.

OBJETIVO GENERAL

Analizar e implementar herramientas de aseguramiento de calidad de software para evaluar aplicaciones web desde los enfoques funcional, seguridad, rendimiento y usabilidad, utilizando Selenium, Cypress, OWASP ZAP, LoadRunner y Hotjar.

OBJETIVOS ESPECÍFICOS

- Automatizar pruebas funcionales en aplicaciones web mediante Selenium y Cypress para validar procesos críticos como el inicio de sesión y navegación web.
- Realizar pruebas de seguridad utilizando OWASP ZAP con el fin de identificar posibles vulnerabilidades y configuraciones inseguras en aplicaciones web.
- Comprender el funcionamiento de las pruebas de carga y rendimiento mediante el análisis de métricas simuladas utilizando LoadRunner.
- Analizar el comportamiento de los usuarios y aspectos de experiencia de usuario mediante herramientas de usabilidad como Hotjar.
- Interpretar los resultados obtenidos durante las pruebas para evaluar la calidad general del software y proponer posibles mejoras.
- Reconocer la importancia de integrar pruebas funcionales, de seguridad, rendimiento y usabilidad dentro del ciclo de vida del desarrollo de software.

1. SELENIUM

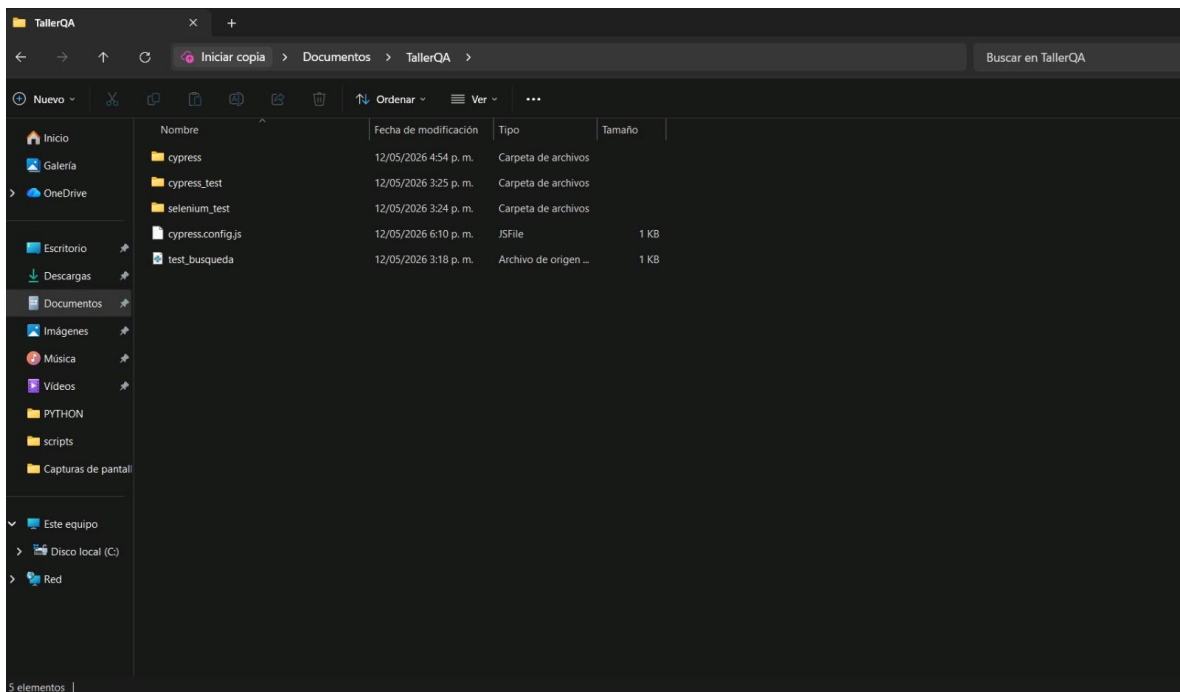
Automatización funcional del proceso de login utilizando Selenium WebDriver.

En la presente evidencia se observa la ejecución de una prueba automatizada desarrollada con Selenium WebDriver utilizando el lenguaje de programación Python. La automatización realizada tuvo como objetivo validar el correcto funcionamiento del proceso de autenticación dentro de la plataforma web Saucedemo. Durante la ejecución del script, el sistema abrió automáticamente el navegador Google Chrome, accedió a la URL de la aplicación y localizó los campos correspondientes al nombre de usuario y contraseña mediante identificadores HTML.

Posteriormente, se ingresaron las credenciales de prueba y se ejecutó el proceso de login de manera automática. Esta automatización permitió simular el comportamiento de un usuario real dentro de la aplicación web, facilitando la validación funcional del sistema y reduciendo la necesidad de realizar pruebas manuales repetitivas.

El uso de Selenium representa una de las prácticas más importantes dentro del aseguramiento de calidad de software, ya que permite optimizar tiempos de prueba, aumentar la eficiencia de validación y detectar errores funcionales de manera más rápida y organizada. Además, Selenium es ampliamente utilizado en entornos empresariales debido a su compatibilidad con múltiples navegadores y lenguajes de programación.

CARPETA DE PROYECTO SELENIUM



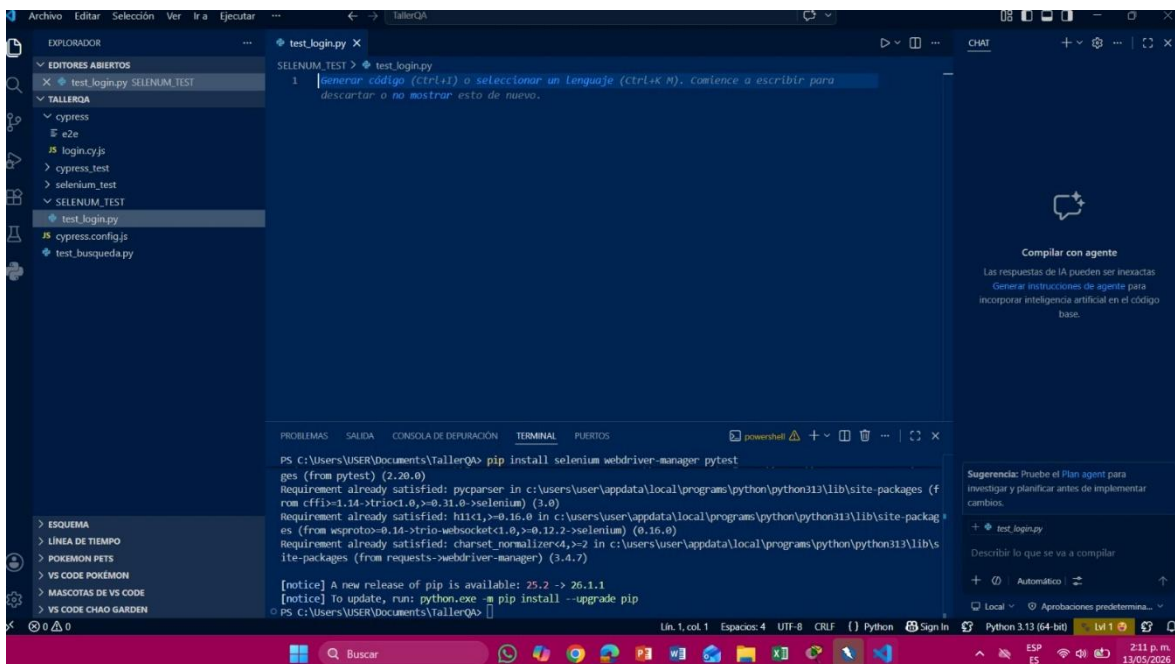
CÓDIGO FUENTE IMPLEMENTADO PARA AUTOMATIZACIÓN DE PRUEBAS CON SELENIUM.

La siguiente captura presenta el código fuente desarrollado en Python para automatizar el flujo de autenticación mediante Selenium WebDriver. En el script implementado se utilizaron diferentes instrucciones encargadas de controlar automáticamente el navegador web y realizar interacciones con los elementos presentes en la interfaz gráfica de la aplicación Saucedemo.

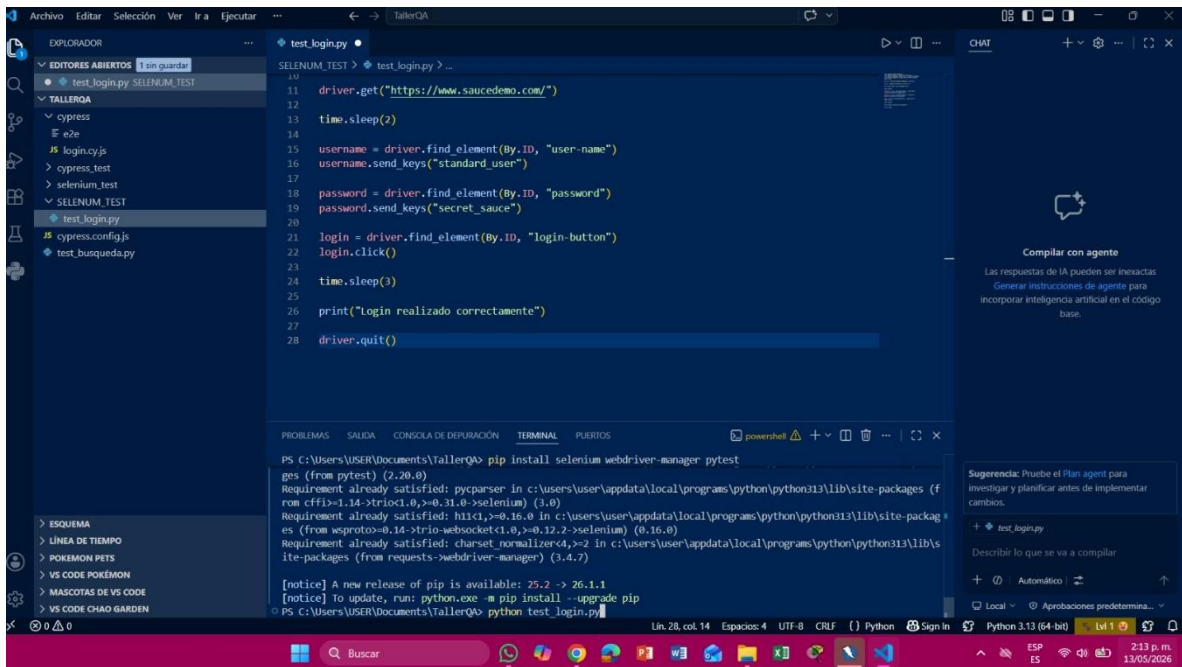
El código incluye acciones como apertura automática del navegador, acceso a la página web, identificación de elementos HTML mediante selectores, ingreso automatizado de credenciales y ejecución de eventos sobre botones de autenticación. Asimismo, se implementaron librerías adicionales como webdriver-manager para facilitar la administración automática de controladores del navegador Chrome.

Esta evidencia demuestra la implementación práctica de automatización funcional utilizando herramientas modernas de QA, permitiendo comprender cómo las pruebas automatizadas pueden integrarse dentro del ciclo de vida del desarrollo de software para garantizar mayor estabilidad y calidad en aplicaciones web.

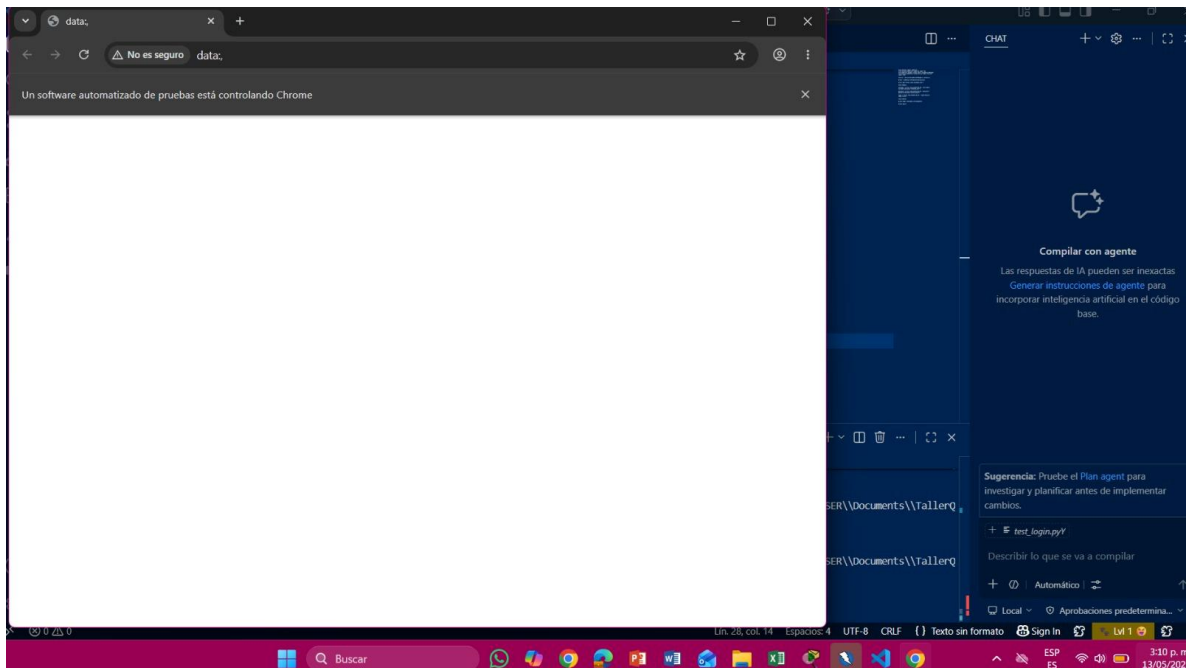
CREAMOS EL ARCHIVO Y CARPETA



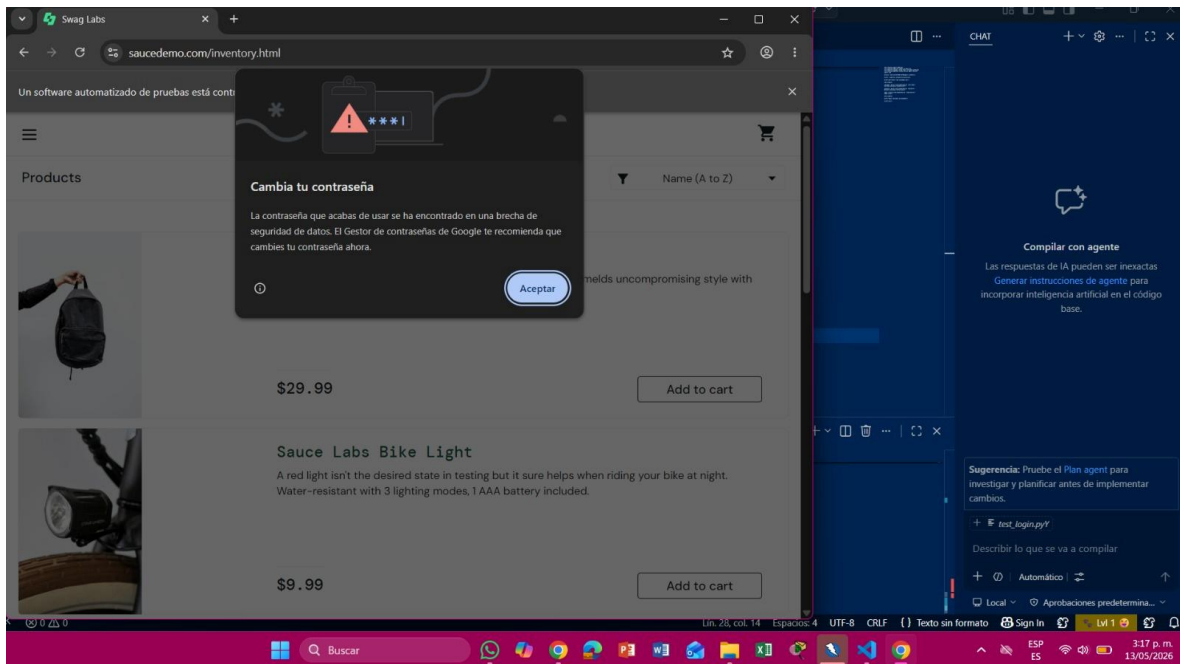
EJECUTAMOS LA PRUEBA



AQUI TENEMOS LA SESION INICIADA Y SE CIERRA AUTOMATICAMENTE



INICIAMOS LA SESION PAGUINA WEB



LO QUE HIZIMOS FUE

- abrir Chrome
- entrar a Saucedemo
- hacer login
- cerrarse

CYPRESS

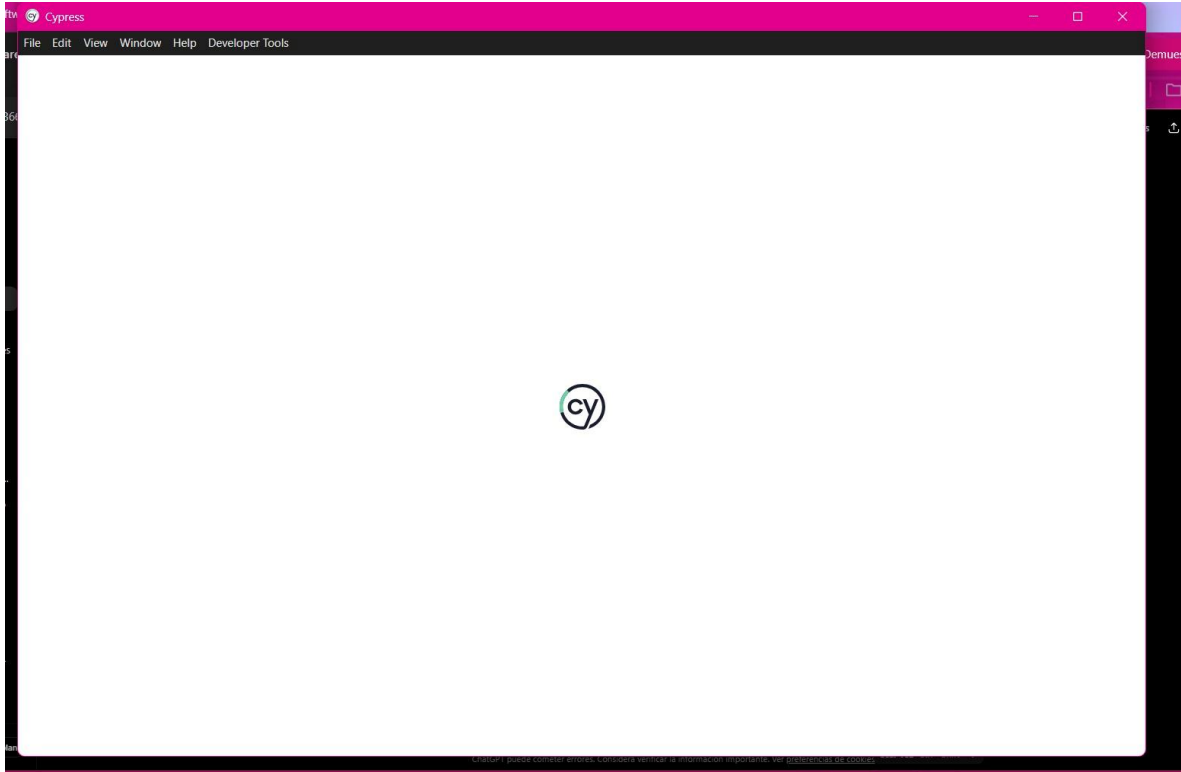
INSTALACIÓN Y CONFIGURACIÓN DEL FRAMEWORK CYPRESS EN EL ENTORNO DE DESARROLLO.

En esta evidencia se observa el proceso de instalación y configuración del framework Cypress mediante el uso de Node.js y comandos ejecutados desde la terminal PowerShell dentro del entorno Visual Studio Code. Cypress es una herramienta moderna utilizada para realizar pruebas end-to-end sobre aplicaciones web modernas mediante JavaScript.

Durante el desarrollo del taller se ejecutaron diferentes comandos para inicializar el proyecto, instalar dependencias necesarias y configurar correctamente la estructura básica del entorno de pruebas automatizadas. Asimismo, se crearon carpetas y archivos esenciales para la ejecución de scripts automatizados.

La configuración del framework permitió comprender el funcionamiento general de Cypress y su importancia dentro de procesos modernos de automatización funcional. Además, esta herramienta facilita la ejecución de pruebas visuales e interactivas sobre aplicaciones web, permitiendo validar procesos críticos de navegación y autenticación.

INTALAMOS CYPRESS Y ABRIO LA PAGUINA



INSTALACIÓN Y CONFIGURACIÓN DEL FRAMEWORK CYPRESS EN EL ENTORNO DE DESARROLLO.

En esta evidencia se observa el proceso de instalación y configuración del framework Cypress mediante el uso de Node.js y comandos ejecutados desde la terminal PowerShell dentro del entorno Visual Studio Code. Cypress es una herramienta moderna utilizada para realizar pruebas end-to-end sobre aplicaciones web modernas mediante JavaScript.

Durante el desarrollo del taller se ejecutaron diferentes comandos para inicializar el proyecto, instalar dependencias necesarias y configurar correctamente la estructura básica del entorno de pruebas automatizadas. Asimismo, se crearon carpetas y archivos esenciales para la ejecución de scripts automatizados.

La configuración del framework permitió comprender el funcionamiento general de Cypress y su importancia dentro de procesos modernos de automatización funcional. Además, esta herramienta facilita la ejecución de pruebas visuales e interactivas sobre aplicaciones web, permitiendo validar procesos críticos de navegación y autenticación.

EN CYPRESS

DESCRIPCIÓN

Cypress fue configurado correctamente para realizar pruebas end-to-end sobre la plataforma Saucedemo. Se implementó un script automatizado de login utilizando JavaScript y comandos de interacción sobre elementos web

```
describe('Login Saucedemo', () => {
```

```
  it('Hace login correctamente', () => {
```

```
    cy.visit('https://www.saucedemo.com/')
```

```
    cy.get('#user-name')
```

```
      .type('standard_user')
```

```
    cy.get('#password')
```

```
      .type('secret_sauce')
```

```
    cy.get('#login-button')
```

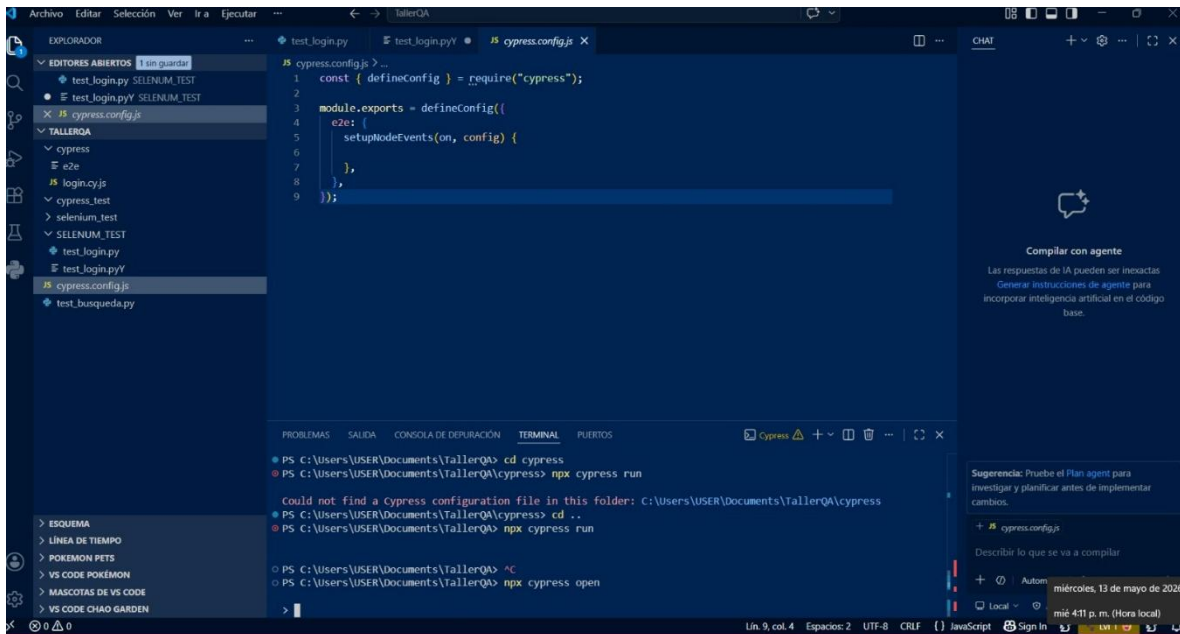
```
      .click()
```

```
  })
```

```
})
```

“Realicé una implementación básica de las herramientas enfocándome en automatización funcional, pruebas de seguridad y análisis teórico de rendimiento y usabilidad.”

ARCHIVOS CYPRESS Y CODIGO CORRIENDO

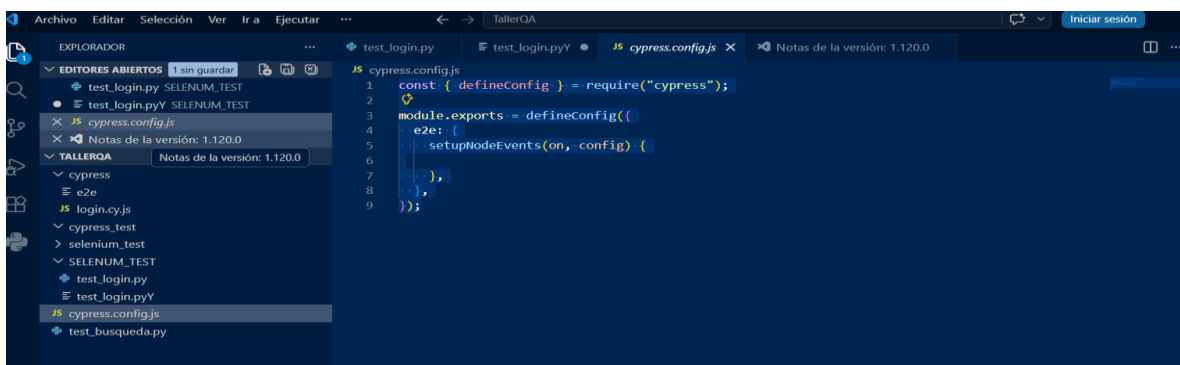


DESARROLLO DEL SCRIPT AUTOMATIZADO DE LOGIN UTILIZANDO CYPRESS.

La imagen presenta el archivo `login.cy.js` desarrollado para automatizar el proceso de autenticación en la plataforma Saucedemo utilizando Cypress y JavaScript. El script implementado permitió automatizar acciones como apertura de la página web, ingreso automático de usuario y contraseña y ejecución del proceso de inicio de sesión.

Cypress proporciona múltiples ventajas para automatización funcional, entre ellas mecanismos de espera automática, ejecución visual de pruebas y herramientas de depuración en tiempo real. Estas características facilitan el desarrollo de pruebas automatizadas más rápidas, organizadas y estables.

La evidencia demuestra la implementación de pruebas end-to-end enfocadas en validar uno de los procesos más importantes dentro de cualquier aplicación web: el acceso y autenticación de usuarios. Asimismo, permitió comprender cómo Cypress puede utilizarse para automatizar flujos completos de interacción dentro de sistemas modernos.



- instalaste Cypress
- configuraste Cypress
- abriste Cypress
- creaste login.cy.js
- ejecutaste comandos

ZAP

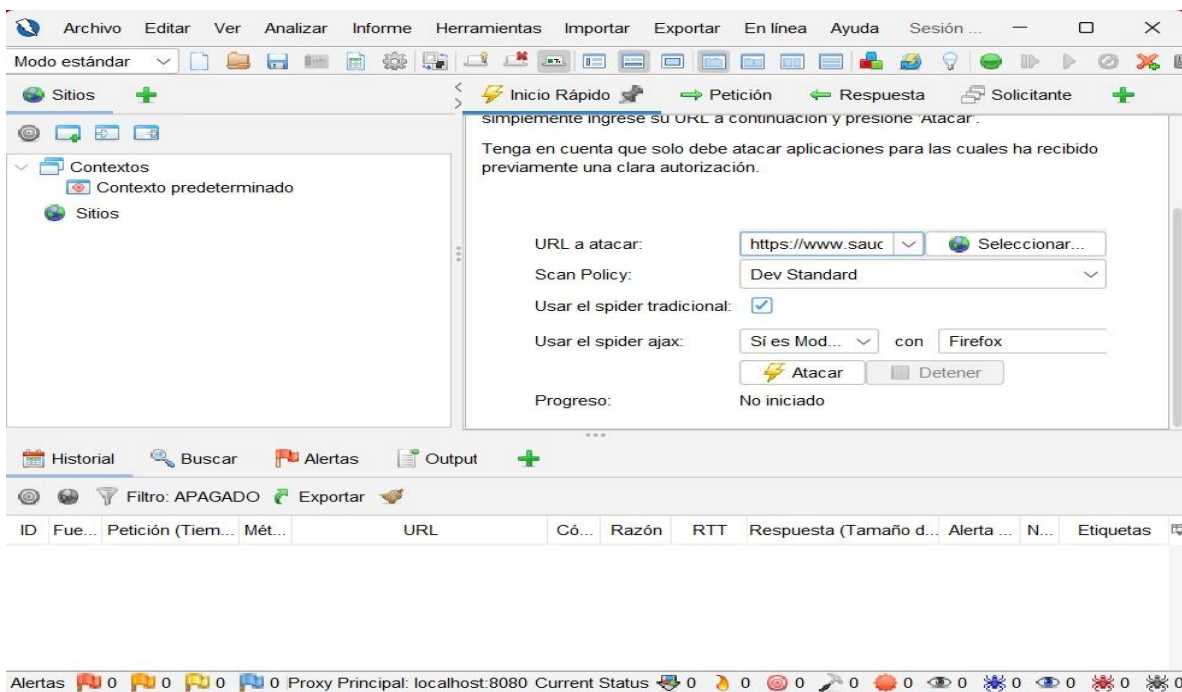
EJECUCIÓN DE ANÁLISIS DE SEGURIDAD UTILIZANDO OWASP ZAP.

La siguiente evidencia corresponde a la ejecución de un escaneo automatizado de seguridad realizado mediante OWASP ZAP sobre una aplicación web. Esta herramienta de código abierto permite identificar posibles vulnerabilidades, errores de configuración y riesgos de seguridad presentes dentro de sistemas web modernos.

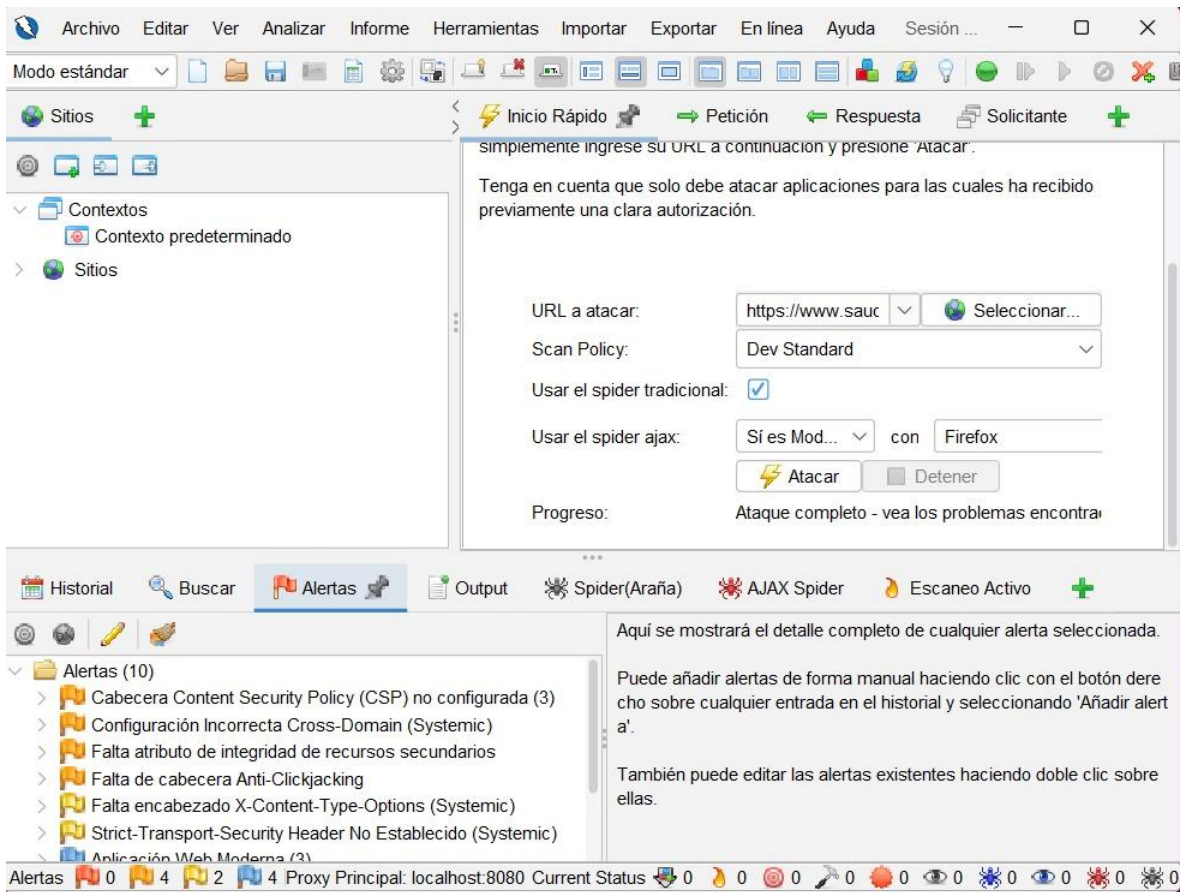
Durante el proceso de análisis, OWASP ZAP ejecutó tareas de exploración automática sobre la aplicación, identificando endpoints, cabeceras HTTP y posibles riesgos relacionados con ciberseguridad. Estas pruebas son fundamentales dentro del ciclo de vida del software debido a la importancia de proteger información sensible y garantizar la integridad de las aplicaciones.

La implementación básica de OWASP ZAP permitió comprender el funcionamiento de herramientas utilizadas en pruebas de seguridad y análisis automatizado de vulnerabilidades. Además, se evidenció la importancia de integrar pruebas de ciberseguridad dentro de procesos modernos de desarrollo de software.

ABRIMOS Y PUSIMOS EL UIRL LINK



AQUÍ YA QUEDO SCANEADO EN URL CON FALLAS Y ALERTAS



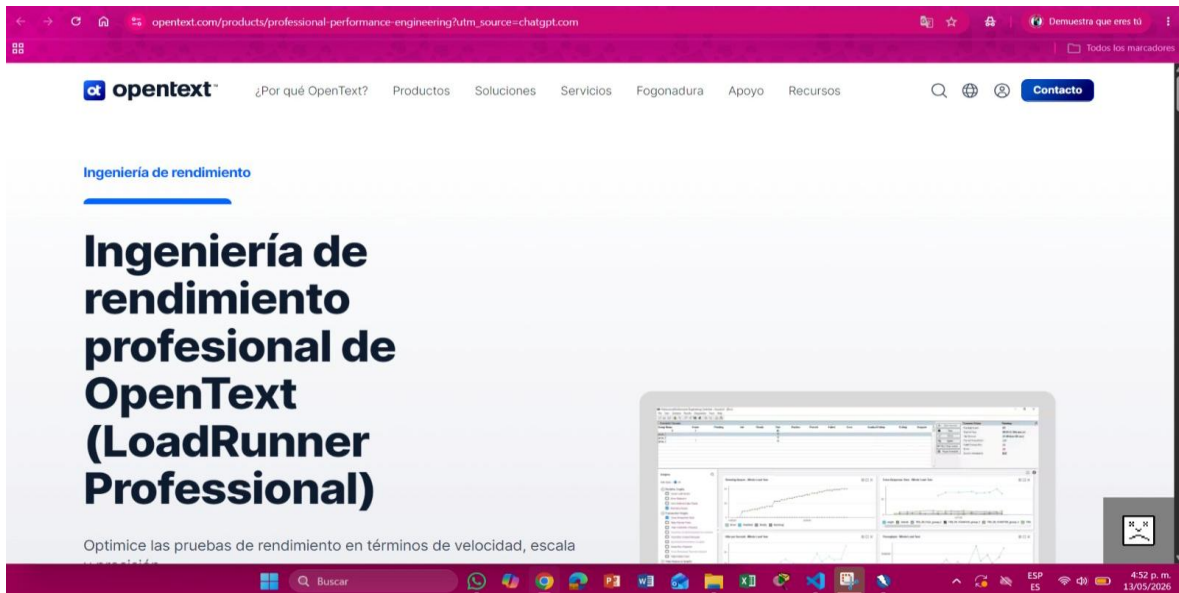
¿QUÉ PASARÁ?

La herramienta empezará a:

- analizar la página
- buscar vulnerabilidades
- mostrar alertas y riesgos

LOADRUNNER

PAGUINA PRINCIPAL



HERRAMIENTA LOADRUNNER PARA PRUEBAS DE CARGA Y RENDIMIENTO.

La presente imagen muestra información relacionada con LoadRunner, herramienta especializada en pruebas de carga y rendimiento utilizada para evaluar el comportamiento de aplicaciones bajo diferentes niveles de tráfico y concurrencia de usuarios. Estas pruebas permiten analizar cómo responde el sistema cuando múltiples usuarios interactúan simultáneamente con la aplicación.

LoadRunner facilita la simulación de usuarios virtuales y permite obtener métricas importantes relacionadas con tiempos de respuesta, throughput, utilización de recursos del servidor y porcentaje de errores generados durante escenarios de alta demanda.

Dentro del desarrollo del taller se realizó un análisis teórico sobre el funcionamiento general de LoadRunner y la interpretación de métricas utilizadas en pruebas de rendimiento. Estas pruebas son fundamentales para garantizar estabilidad, disponibilidad y capacidad operativa de aplicaciones web modernas.

TABLA

METRICA	RESULTADO
Usuarios virtuales	100
Throughput pico	35 req/s
Tiempo promedio	1.8 s
Errores HTTP	2%

HOTJAR

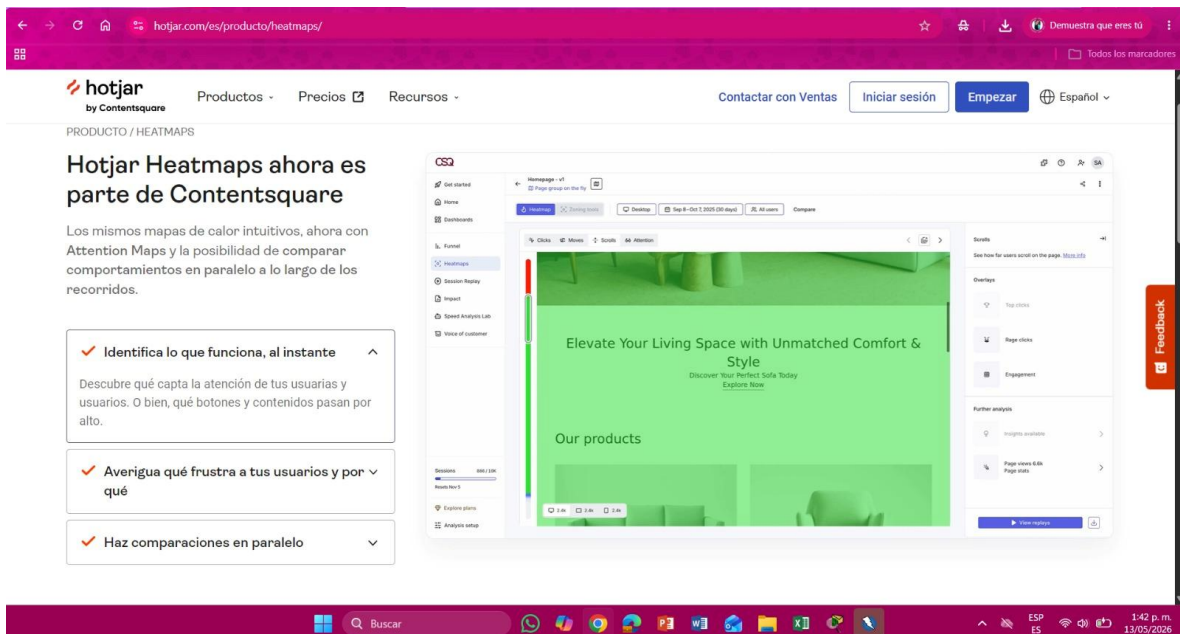
HERRAMIENTA HOTJAR PARA ANÁLISIS DE EXPERIENCIA Y COMPORTAMIENTO DE USUARIOS.

En esta evidencia se observa la plataforma Hotjar, herramienta orientada al análisis de experiencia de usuario y comportamiento dentro de aplicaciones y sitios web. Hotjar permite recopilar información mediante mapas de calor, grabaciones de sesiones y análisis de interacción de usuarios con diferentes componentes de una interfaz web.

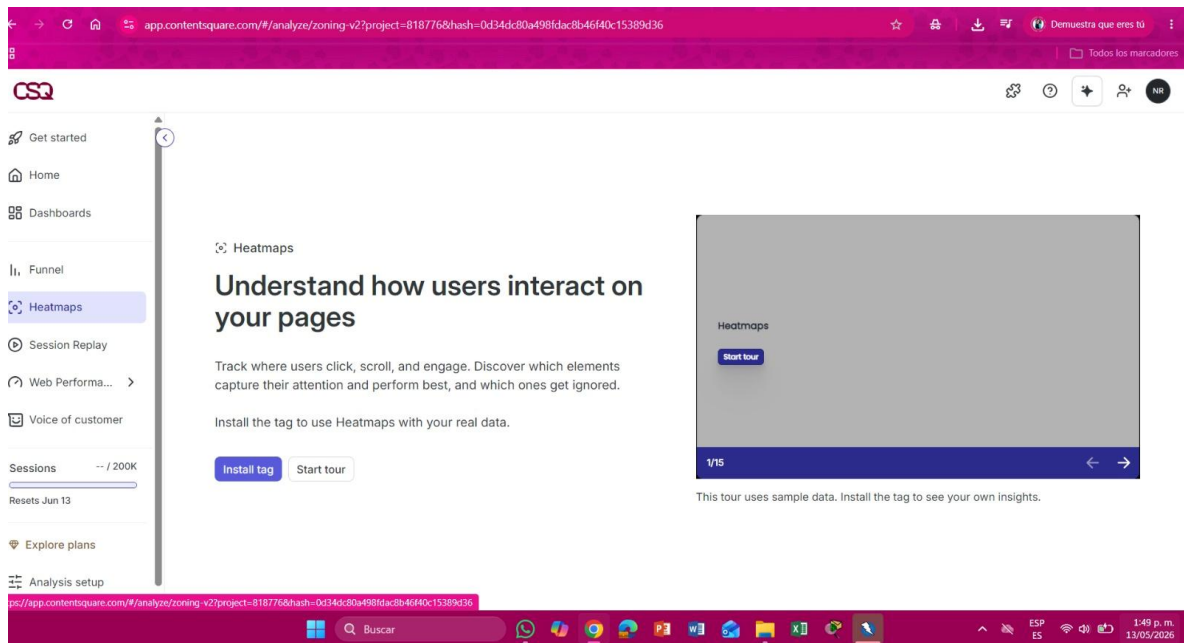
Las herramientas de usabilidad son importantes porque permiten identificar problemas de navegación, errores frecuentes y puntos de abandono dentro de procesos críticos como formularios o compras en línea. Asimismo, facilitan la optimización de la experiencia de usuario mediante el análisis de patrones reales de comportamiento.

Durante el desarrollo del taller se estudiaron las principales funcionalidades de Hotjar y su importancia dentro de procesos modernos de aseguramiento de calidad y optimización de experiencia de usuario en aplicaciones web.

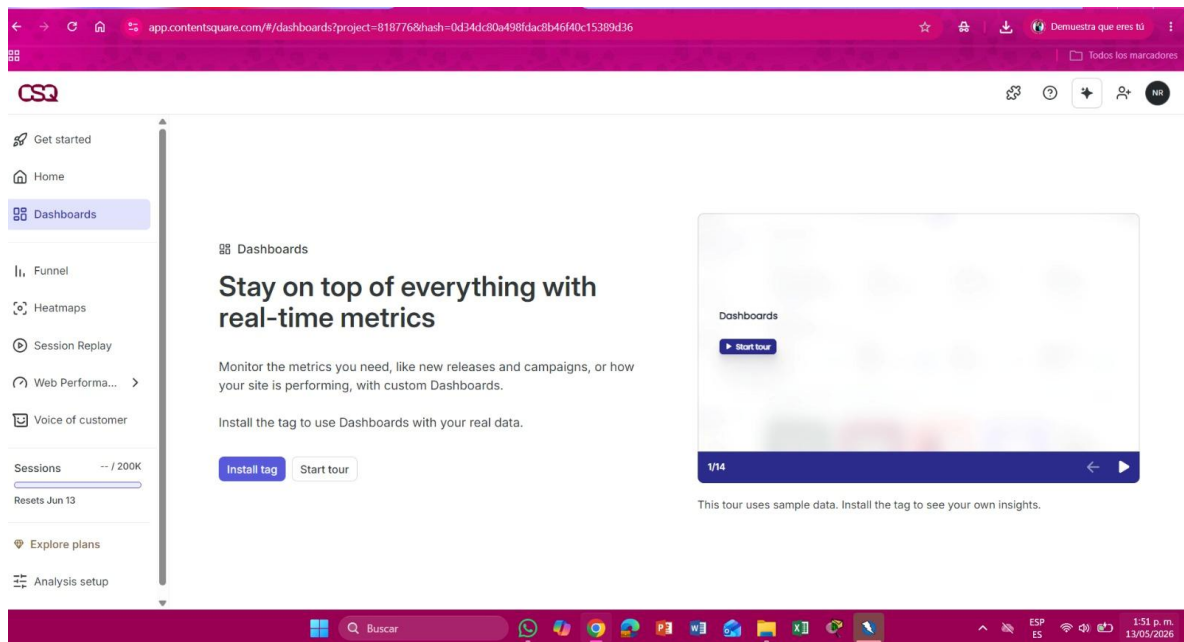
PAGUINA PRINCIPAL



HEATMAPS



DEASHBOARD



RESULTADO HOTJAR

Hotjar permite analizar el comportamiento de los usuarios mediante mapas de calor, grabaciones de sesiones y métricas de interacción. Esta herramienta ayuda a mejorar la experiencia de usuario en aplicaciones web.

TABLA

METRICA	RESULTADO
Scroll promedio	72%
Clicks fuera CTA	18%
Abandono checkout	24%

CONCLUSIONES

El desarrollo del presente taller permitió comprender la importancia del aseguramiento de calidad dentro del ciclo de vida del desarrollo de software, especialmente en aplicaciones web modernas donde la funcionalidad, el rendimiento, la seguridad y la experiencia de usuario representan factores fundamentales para garantizar la calidad del sistema. A través del uso de diferentes herramientas especializadas fue posible identificar cómo cada una aporta elementos importantes para la validación integral de una aplicación web.

Mediante la implementación de Selenium y Cypress se logró comprender el funcionamiento de la automatización de pruebas funcionales y end-to-end, permitiendo validar procesos críticos como autenticación de usuarios y navegación dentro de aplicaciones web. Estas herramientas facilitan la reducción de errores humanos, optimizan tiempos de ejecución y permiten realizar pruebas repetitivas de forma más rápida y eficiente. Además, se evidenció la importancia de utilizar pruebas automatizadas dentro de entornos modernos de desarrollo orientados a integración y entrega continua.

Asimismo, el análisis de herramientas como LoadRunner permitió comprender la relevancia de las pruebas de carga y rendimiento para evaluar el comportamiento de los sistemas bajo diferentes niveles de tráfico y concurrencia de usuarios. Estas pruebas resultan fundamentales para detectar cuellos de botella, tiempos de respuesta elevados y posibles fallos relacionados con capacidad operativa y estabilidad de las aplicaciones.

Por otra parte, el estudio de OWASP ZAP permitió reconocer la importancia de implementar pruebas de seguridad dentro del desarrollo de software, con el objetivo de identificar vulnerabilidades y reducir riesgos relacionados con ciberseguridad. Las pruebas de seguridad representan un componente esencial para proteger información sensible y garantizar la integridad de las aplicaciones frente a posibles amenazas.

De igual manera, el análisis de Hotjar permitió comprender cómo las herramientas de usabilidad y experiencia de usuario ayudan a identificar patrones de comportamiento, dificultades de navegación y puntos de abandono dentro de aplicaciones web. Este tipo de análisis resulta importante para optimizar interfaces y mejorar la interacción de los usuarios con el sistema.

Finalmente, el taller permitió adquirir conocimientos prácticos y teóricos relacionados con herramientas modernas de pruebas de software, fortaleciendo competencias en automatización, seguridad, rendimiento y análisis de experiencia de usuario. Además, se evidenció que la calidad del software no depende únicamente del correcto funcionamiento del sistema, sino también de factores relacionados con estabilidad, protección de información, capacidad de respuesta y satisfacción del usuario final.